

Phase 2: Requirement Analysis — Solution Requirements Specification

Project Name: Cab Booking (`UCab`)

Project ID: `N/A (Solo Track Submission)`

Team Member / Solo Developer: Shaik Sumiya Zainab

1. Functional Requirements Specification

1.1 Authentication & Authorization Module (`FR-AUTH`)

- `FR-AUTH-01` (Rider Registration): The system must allow users to register by submitting their full name, email address, password, and phone number.
- `FR-AUTH-02` (Rider & Admin Login): The system must authenticate users via `bcrypt` password comparison and issue a JSON Web Token (`JWT`) with a 30-day expiration (`protect` middleware).
- `FR-AUTH-03` (1-Click Demo Login): To facilitate rapid evaluation and grading, the login screens (`Login.jsx` and `AdminLogin.jsx`) must feature one-click buttons that instantly populate credentials and establish authenticated sessions without manual typing.
- `FR-AUTH-04` (Role-Based Route Gating): The frontend and backend must strictly enforce access control (`userRole === 'admin'`), preventing standard riders from accessing executive KPI or fleet management routes (`/admin/\*`).

1.2 Fleet Catalog & Discovery Module (`FR-FLEET`)

- `FR-FLEET-01` (Multi-Category Categorization): The system must classify cab inventory into 4 distinct vehicle classes:
 - Mini: Compact hatchbacks (`110/km`, Base: `140`, 4 seats).
 - Sedan: Executive sedans (`112/km`, Base: `150`, 4 seats).
 - SUV: Spacious multi-utility vehicles (`118/km`, Base: `180`, 6 seats).
 - Luxury: Premium sedans (`135/km`, Base: `150`, 4 seats).
- `FR-FLEET-02` (Interactive Filtering & Sorting): The client must allow users to filter available cabs by category tabs and sort vehicles dynamically by rate per kilometer (`1/km`).

1.3 Trip Booking & Fare Calculation Engine (`FR-BOOK`)

- `FR-BOOK-01` (Deterministic Distance & Duration Calculation): When a user inputs pickup and drop-off locations, the system must calculate the trip distance (`km`) and dynamically estimate trip duration (`minutes = Math.ceil(distance \* 2.5)`).
- `FR-BOOK-02` (Real-Time Fare Computation): The total trip cost must be calculated instantaneously using the formula:
$$\text{Total Fare} = \text{Base Fare} + (\text{Distance in KM} \times \text{Price Per KM}) - \text{Promo Discount} + \text{Add-On Perks}$$
- `FR-BOOK-03` (Promo Code Engine): The system must validate discount codes (`UCAB20` for 20% off and `WELCOME10` for 10% off) and reflect exact deduction badges in real time.
- `FR-BOOK-04` (Perks & Customization Suite): Users must be able to toggle optional checkboxes during checkout:
 - In-Ride Refreshments Package (`+ 150`): Chilled mineral water and snacks upon pickup.
 - Road Safety Charity Contribution (`+ 120`): Direct donation to driver healthcare welfare funds.

1.4 Real-Time Dispatch Tracking & Invoicing (`FR-TRACK`)

- `FR-TRACK-01` (Interactive GPS Progress Bar): Upon booking confirmation, the user must be redirected to a live dispatch tracker (`UserHome.jsx`) visually indicating 4 stages: `Booking Accepted`, `Driver Assigned`, `Cab in Transit`, and `Completed`.

- `FR-TRACK-02` (Driver Credential Injection): The system must assign a verified driver profile (`Vikram Sharma - 4.9 & `, contact number, and vehicle registration plate) to every accepted booking.
- `FR-TRACK-03` (Corporate PDF Thermal Invoices): Users must be able to view their complete trip history (`MyBookings.jsx`) and download formal corporate/thermal tax invoices featuring an embedded QR Verification Code (`ReceiptModal.jsx`).

1.5 Executive Admin Control Panel (`FR-ADMIN`)

- `FR-ADMIN-01` (Analytics KPI Dashboard): The system must aggregate and display top-level metrics: Total Gross Revenue (`<sup>1</sup>`), Total Bookings Count, Active Registered Users, and Operational Fleet Count.
- `FR-ADMIN-02` (Fleet CRUD Operations): Dispatch administrators must be able to create (`AddCar.jsx`), read (`AdminCars.jsx`), update (`EditCar.jsx`), and delete cab records with image upload capability.
- `FR-ADMIN-03` (System-Wide Trip Dispatch Control): Administrators must be able to monitor all platform bookings (`AdminBookings.jsx`) and update any trip status (`Pending` ! `Accepted` ! `Started` ! `Completed`) with immediate client sync.

2. Non-Functional Requirements Specification (`NFR`)

- `NFR-01` (Zero-Config Database Resilience): The application must never crash due to a missing local database daemon. The backend (`config.js` and `store.js`) must detect if MongoDB is unreachable within 2 seconds and automatically fall back to an atomic JSON storage engine (`data.json`).
- `NFR-02` (UI/UX Visual Excellence): The interface must utilize modern glassmorphism aesthetics, curated typography (Plus Jakarta Sans), smooth CSS transitions, and responsive grid patterns (`index.css`).
- `NFR-03` (Security & Data Integrity): All user passwords must be hashed using `bcrypt` with salt rounds before storage. API requests to protected endpoints must carry valid Bearer JWT tokens in the `Authorization` header.